



INDIVIDUAL CAPSTONE **REPORT**

Title: A Decentralized Freelance Protocol with Web3 Integration

Date: 03/04/2026

April - 2026



Title: Decentralized Freelance Protocol with Web3 Integration

Date: 2026/04/03

General Information

Project Title: Decentralized Freelance Protocol with Web3 Integration

Points of Contact & Project Members

Position	Full Name	Student ID	Roles & Responsibilities (Brief)	E-mail
Supervisor	Subit Timalina	—	Supervision	subit@iimscollege.edu.np
Team Lead	Sarun Mahrajan	0371759	Project Manager / Backend & Blockchain Developer	sarun2080-0228@iimscollege.edu.np
Member 1	Bijee Dangol	0372053	Frontend Developer	bijee2080-212@iimscollege.edu.np
Member 2	Pawan Poudel	0371478	Backend Database & Storage Specialist	pawan2080-0296@iimscollege.edu.np
Member 3	Anushree Pradhan	0372148	Backend, API Developer, Database Assistant	anushree2080-0344@iimscollege.edu.np
Member 4	Runa Mapu	0371373	System Workflow Engineer, Junior Backend Developer	runa2080-0189@iimscollege.edu.np

Chapter 1:Project Introduction.....	4
Chapter 2: Related Works.....	4
2.1 Overview.....	4
2.2 Methodology.....	5
2.2.1 Technology Analysis.....	5
2.3 Summary of Literature.....	7
Summary Table of Related Works (Individual API & Data Focus).....	7
2.4 Trends, Gaps, and Conclusion.....	8
3.2 Technical Analysis.....	11
3.3 Requirement Specifications.....	11
4.1.1 System Components.....	13
4.1.2 Data Flow & Communication.....	14
4.1.3 Technology Stack.....	14
4.2 Use Case Diagram.....	15
4.8 Pseudocode and Algorithmic Structure.....	24
4.8.1 Core Functionality: ECDSA Authentication Logic.....	24
4.8.2 Database Transactions: IPFS Metadata Mapping.....	26
4.8.3 Logic for State Synchronization (Blockchain Listener).....	26
References.....	34

Chapter 1:Project Introduction

Chapter 1 is already submitted but there are some additions in the roles and responsibilities.

1.5.3 Roles and Responsibilities

Member Anushree	Previous	New
Anushree Pradhan	Backend API Engineer	Backend API and Centralized Database Engineer

Chapter 2: Related Works

2.1 Overview

This shift from a centralized platform for freelancers (Web2) to a decentralized platform (Web3) is driven by necessity. Past P2P marketplaces were plagued by latency issues and data silos. Current studies have identified that the only viable solution for business viability is Hybrid Architecture. As the API and Data Engineer, my role is centered on the "High-Performance Bridge" using FastAPI and PostgreSQL to keep in sync with blockchain state.



Figure: Evolution of Web

2.2 Methodology

The methodology is informed by three technical pillars identified in recent literature:

- Asynchronous API Orchestration: Sharma, A., & Gupta, R. (2025) have shown that FastAPI's non-blocking I/O operation cuts the communication overhead of Web3 by 30%. This guides our decision to use a Python-based asynchronous technology stack to ensure our Auth Latency is always less than 1.5s even for a decentralized node polling operation.
- Relational Decentralized Indexing: Pippal & Sharma (2025) have proved that although IPFS provides "Content-Addressability," it does not offer any facility for searchability. Their research validates our decision to use PostgreSQL to index CIDs (Content Identifiers), allowing our users to use complex SQL queries (filters/search) on immutable data.
- Cryptographic Identity Management: Singh et al. (2025) and Careja et al. (2026) have shown that if ECDSA (Elliptic Curve Digital Signature Algorithm) is deployed at the API entry point, it eliminates the "Single Point of Failure" of password databases. This research-based approach allows us to use MetaMask for "Passwordless" login.

2.2.1 Technology Analysis

The selection of the tools to use in implementing the methodologies mentioned above is justified in this section with emphasis on performance and data integrity.

- FastAPI (Backend Framework):

FastAPI is based on ASGI (Asynchronous Server Gateway Interface). In a Web3 protocol, there are high latency tasks such as IPFS pinning and blockchain mining, which the protocol needs to wait for. Asynchronous FastAPI can deal with these so-called wait-states without blocking the server, which is a condition to reach our goal of Auth Latency = 1.5s.

Alternative: Express.js (Node.js). In spite of being a highly trending option, Node.js is not able to rival the Python-based Web3 libraries present, including web3.py, which offer better cryptographic recovery functionality.

- PostgreSQL (Metadata Indexing):

ACID compliance in financial transactions is required in this project, which can be offered by PostgreSQL.

Alternative: MongoDB. Although MongoDB is a very versatile database, NoSQL cannot provide relational mapping to ensure a Job ID is perfectly synced with a particular IPFS hash and Smart Contract address.

- Web3.py & Ethers.js Bridge:

These libraries are the center of our "Digital Wax Seal." They help the API to reverse-match Ethereum addresses and digital signatures. It is through this that our passwordless identity pipeline will be affected.

- Pydantic (Data Validation):

Our Data Firewall is Pydantic. It strictly implements type hints. It blocks corrupted data, like a wallet address of the wrong size or a CID string of the wrong format, into the API. It shields the PostgreSQL backend from data corruption.

- Kubo (IPFS) storage engine:

Kubo is the infrastructure that is necessary to support the project deliverables. Through the Kubo library, the API would be able to make sure that after the project is posted, it would not be changed by the client or the freelancer.

2.3 Summary of Literature

Summary Table of Related Works (Individual API & Data Focus)

Author(s) & Year	Paper Title / Topic	Methodology / Tools	Key Contributions	Relevance to your Role
Hwang et al. (2026)	AI Disclosure in Freelance Work	Mixed-Methods / AI Governance	Explores how decentralized infrastructures support trust in gig economies.	Informs the Metadata Schema for AI-usage flags in your PostgreSQL DB.
Kadam et al. (2024)	Analysis of Decentralized Freelancing	Ethereum / P2P Logic	Analyzes two-way confirmation logic for task completion.	Defines the API State Machine for job "Confirmation" status.
Careja et al. (2026)	Digital Identity Using Blockchain	ECDSA / Cryptographic Sig.	Proposes a model for secure decentralized management of digital identity.	Validates your FastAPI Auth pipeline using wallet signatures instead of passwords.
Singh et al. (2025)	Blockchain for Enhanced API Security	Automated Policy Enforcement	Demonstrates 67% reduction in unauthorized access using blockchain-secured APIs.	Justifies your JWT + Blockchain hybrid authentication approach.
Singh et al. (2025)	Improved ECDSA Batch Verification	Hamming Weights / Batch Sig.	Optimizes signature verification speed to reduce system latency.	Directly informs your goal of < 1.5s Auth Latency for API calls.

Boonchieng et al. (2025)	Super APIs for Blockchain Apps	Cross-Chain Protocol / AES	Designs high-performance APIs for financial transactions in DApps.	Provides a blueprint for scalable API request-response flows.
Pippal & Sharma (2025)	Infrastructure Hybrid Blockchain IPFS	Content-Addressing / IPFS	Evaluates using IPFS for distributed data while maintaining an on-chain hash.	Informs your Data Engineering task of mapping IPFS CIDs to PostgreSQL.
Singh et al. (2025)	Trustless Marketplaces and AI	AI-Blockchain Integration	Shows that trustless platforms reduce transaction fees by 65%.	Highlights the need for Automated Escrow APIs in your backend.
Ren et al. (2025)	Scaling Metadata Performance	IndexFS / Caching	Discusses how middleware improves metadata and small file operation efficiency.	Informs your use of PostgreSQL as a cache/index for IPFS data.
Gupta et al. (2024)	Blockchain Scalability and Optimization	Snake Optimization / Latency	Focuses on reducing latency in decentralized transaction processing.	Supports the performance benchmarking of your registration APIs.

2.4 Trends, Gaps, and Conclusion

- **Trends:**

The literature has distinctly moved towards Metadata Caching through the use of a centralized database such as PostgreSQL as a high-speed cache of the slow and immutable ledger.

- **Gaps:**

The majority of the current studies have neglected Atomic Synchronization. The majority of the research has not covered the aspect of a database state being rolled back in case one of the blockchain transactions (such as a deposit in escrow) fails.

- **Conclusion:**

This project aims to fill these gaps by introducing a Deterministic State Sync logic in the FastAPI layer. With PostgreSQL Relational Integrity and IPFS Content Addressability, we can develop a system that is as fast as a Web2 system and as safe as a Web3 system. This is a direct response to the content linking inadequacy issues highlighted in our defense of the research.

Chapter 3: System Analysis

3.1 Theoretical: SWOT Analysis

Being the member in charge of the Middle Layer of the protocol, this analysis will assess the internal and external forces on the creation of the high-performance API bridge.



The following table is the analysis of the API Backend and Data stack.

Feature	FastAPI (Selected)	Express.js (Alternative)	PostgreSQL (Selected)	MongoDB (Alternative)
Strengths	Native async support; Pydantic type safety; built-in Swagger UI.	Huge library ecosystem; widely used in Web3.	ACID compliance; robust relational indexing for CIDs.	High performance for unstructured data.
Weaknesses	Smaller middleware community than Node.js.	Single-threaded; prone to "callback hell" in complex syncs.	Rigid schema; requires migrations for updates.	Lack of joins makes transaction tracking difficult.
Opportunities	Seamless integration with Python's Web3 and AI libraries.	Universal language (JS) for full-stack consistency.	JSONB support allows for "Hybrid" data flexibility.	Rapid prototyping for non-relational data.
Threats	Breaking changes in fast-moving Python dependencies.	Performance bottlenecks during heavy cryptographic hashing.	Performance lag in very large datasets without sharding.	Data consistency risks in financial escrow logic.

- FastAPI vs Express: This project requires the permanent communication with our Kubo (IPFS) node and Hardhat blockchain. It is a wait mechanism of high latency. FastAPI is asynchronous, and this implies that our API can continue to work until a file is pinned or a transaction is mined. This will be significant to our 1.5s Auth Latency objective.
- Free Free-lance data refer to relational data. User-Job- Deliverable- Payment. There cannot be orphaned data. A case in point is where one has a payment and no relationship with a deliverable. PostgreSQL is compatible with the ACID enabling us to guarantee that our Digital Wax Seal (the relationship between our Job and our IPFS hash) is never lost.

3.2 Technical Analysis

This section shows the details of the Logic and Infrastructure from a backend perspective.

- Logical Algorithms: The "State-Machine" Escrow
 - Use a deterministic state-transition logic. The API validates that a state change (e.g.; from In-Progress to Submitted) only happens if a valid IPFS CID is provided and verified.
- Core Libraries & Tools

Technical Tool	Category	Primary Function in Architecture	Justification
Web3.py	Blockchain Middleware	Facilitates the "Digital Wax Seal" by recovering Public Keys from ECDSA signatures.	Replaces centralized password storage with Sovereign Identity, ensuring zero-knowledge authentication.
Kubo (IPFS)	Distributed Storage	Acts as the primary node for Content-Addressable Storage (CAS) of deliverables.	Ensures file immutability; the generated CID acts as a permanent cryptographic receipt of the work.
Pydantic	Data Validation	Enforces strict Type-Hinting and Schema validation for incoming API payloads.	Prevents malformed data from reaching the database, mitigating Injection attacks and ensuring 100% data integrity.

3.3 Requirement Specifications

3.3.1 Functional Requirements

- Identity Orchestration:
 - The system ought to accommodate a registration and authentication system on the basis of user Public Wallet Address and a digital signature.
- Metadata Indexing:

- The API will allow searching APIs in job postings with the ability of the user to search by budget, skills, and time stamps which are accessed in the PostgreSQL database.
- Storage Handshake:
 - The system is expected to enable communication with the Kubo (IPFS) node in which the deliverables are stored and the CID (Content Identifier) obtained is stored in the relational database.
- Transaction Synchronization:
 - The API has to be used to provide event-driven synchronization service offered by the Solidity Smart Contract.

3.3.2 Non-Functional Requirements

- Performance (Auth Latency):

According to the defense objectives, the API must be capable of making the ECDSA signature recovery and giving a session token in 1.5 seconds.
- Security (Data Integrity):

Any metadata stored in PostgreSQL and referring to a file must be linked to an unchangeable CID so that the backend does not reference the content that is being modified or deleted.
- Scalability:

The API must be developed based on the concepts of RESTful and stateless so that the backend can be containerized, i.e. run on Docker, and be scaled horizontally.
- Availability:

The storage bridge must also have a retry mechanism in place to IPFS pin operations so that the additions of deliverables in the network can be performed even during the outage of a node.

Chapter 4: System Design

In this chapter, particular attention will be paid to the system design. The system design will be carried in a manner that entails the use of visual representation and diagrams to make the system implementation process clear and efficient. System design is the stage where the outline of the system is outlined.

4.1 System Architecture Diagram

It is an architecture that is founded on the Hybrid Decentralized Model. It makes use of a centralized high-speed API which allows the user experience of the Web2 application with the safety and irrevocability of the Web3 infrastructure.

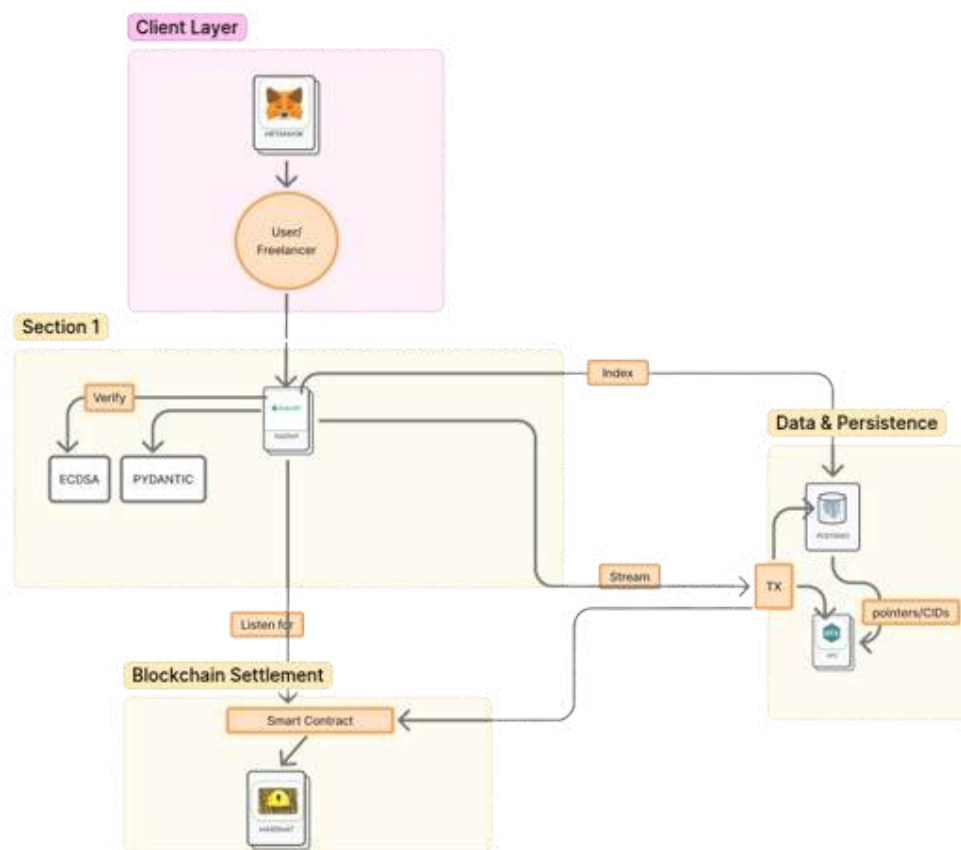


Fig: System Architecture Diagram

4.1.1 System Components

The system consists of four functional levels that control the lifecycle of a freelance contract and the API & Data Engineer divides the system into these levels:

Client Layer (Frontend): This layer will be the user interface layer and will be coded on either React or Flutter. This layer will be used thereof to communicate with the backend. This layer will also involve the integration of the client-side wallet and the use of MetaMask to sign the nonce to do the initial authentication.

Orchestration Layer (FastAPI Backend): This layer will form the Brain of the system. This layer will also consist of the routing of the requests, Pydantic validation, and ECDSA signature recovery.

Persistence & Storage Layer (PostgreSQL & Kubo): This will be a two-component system. The structured and querying metadata (job titles, budgets, user status, etc.) will be stored in PostgreSQL. The unorganized and unchangeable data (project files, deliverables, etc.) will be saved in Kubo with the assistance of IPFS.

Settlement Layer (Hardhat/Blockchain): This layer will be the Financial Truth layer. The Solidity Smart Contracts on the escrowed payments will also be included in this layer. This backend will be subscribing to the events that this layer generates in order to update the status in the database.

4.1.2 Data Flow & Communication

The data flow is intended to be Asynchronous and Event-Driven to ensure that the system is responsive:

- Identity Handshake: The information sent between the User Interface and Fast API Gateway is a signed message. Here, the backend retrieves the public key and authenticates the identity without a password database.
- Metadata Indexing: Once the identity is verified the job information is sent to PostgreSQL by the processing engine. In this case, the engine sends the user a Job_ID as a direct response.
- Storage Hand shake: Once the freelancer uploads his or her file, the API forwards the information to the Kubo IPFS Node. In this case, IPFS will provide the API with a CID (Content Identifier), which will be pinned to the latter by the API to the desired row in the PostgreSQL database.
- State Synchronization: The backend uses an Event Listener to observe Hardhat External API. Once event Payment Released is identified then information is fed back to the database to update the status to Completed.

4.1.3 Technology Stack

The following technologies were selected based on their ability to handle high concurrency and cryptographic operations:

Component	Technology	Selection Justification
Programming Language	Python 3.10+	Offers superior libraries for Web3 (web3.py) and high-speed data processing.
Backend Framework	FastAPI	Asynchronous (ASGI) support allows for non-blocking I/O during IPFS/Blockchain calls.
Primary Database	PostgreSQL	Provides ACID compliance necessary for tracking financial metadata and relational integrity.
Decentralized Storage	Kubo (IPFS)	Ensures project deliverables are content-addressable and immutable.
Validation Layer	Pydantic	Enforces strict data schemas to prevent malformed requests from reaching the database.
Web3 Bridge	Web3.py	Facilitates the ECDSA signature recovery and smart contract interactions.

- System Components: Identify the main components of the system, which may include the front-end, back-end, database, and AI/ML processing units.
- Data Flow & Communication: Identify how data flows between different levels or components (user interface, processing unit, database, and external APIs).
- Technology Stack: Briefly describe the technology stack used for each component.

4.2 Use Case Diagram

The diagram shows the boundaries within which the system functions based on the API interface. Although my team is responsible for the overall interface, my role is specifically focused on the Identity and Storage handshakes.

4.2.1 Primary Actors

The Freelancer and Client are the actors for the overall system, while the FastAPI Backend is the primary actor for automating these interactions.

4.2.2 Key Use Cases

- Cryptographic Authentication
 - This is different from normal authentication as the API will have to recover a public key based on a signature received from MetaMask.
- Secure File Pinning
 - This represents my role in ensuring that data is sent to a decentralized storage interface while immutably linking the user profile to a received CID.
- Event Monitoring
 - This is an automated use case where the API will listen for events on the Hardhat blockchain.

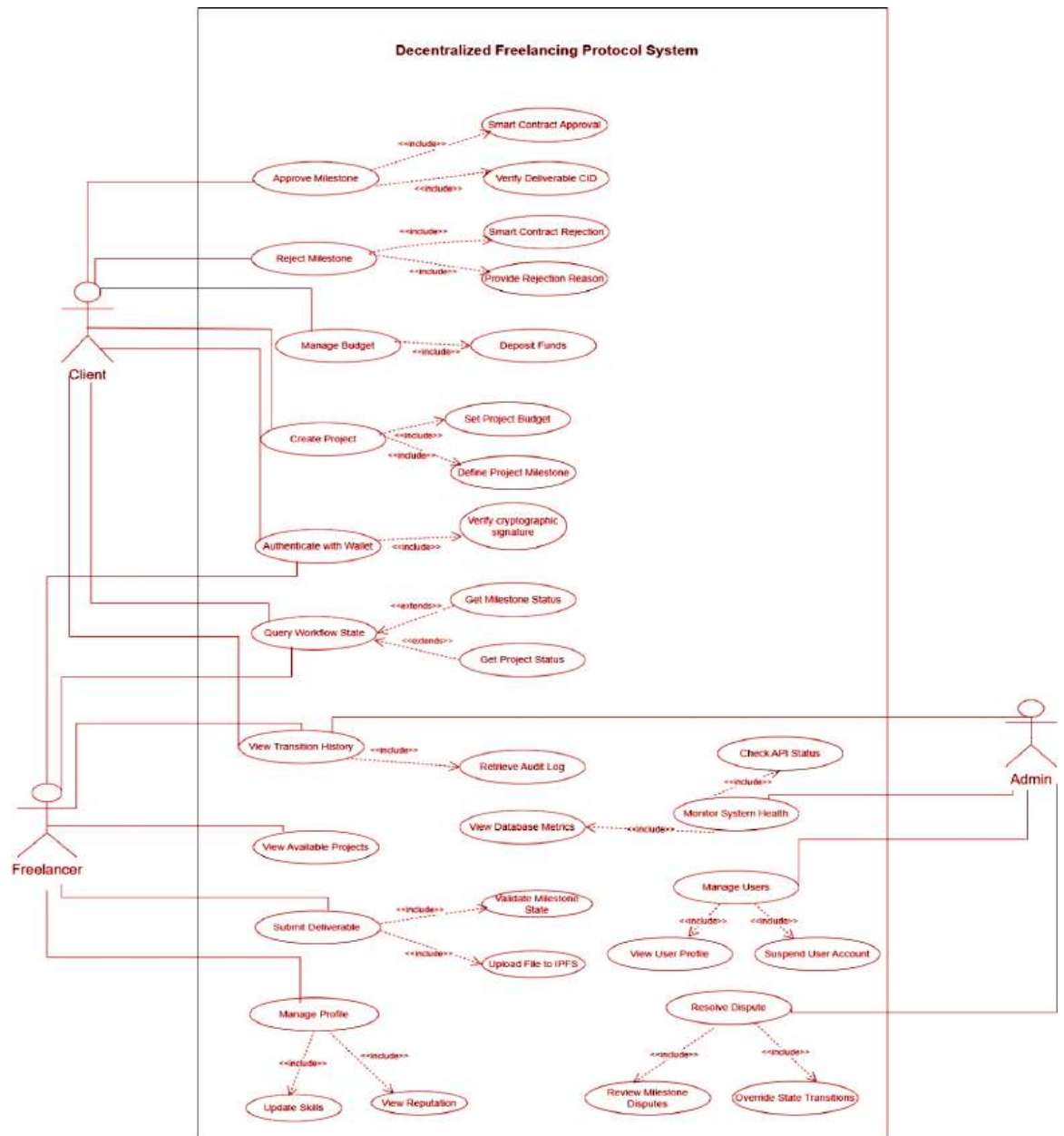


Fig: Use Diagram for decentralized Freelance Protocol with Web3 Integration web3

4.6 Class Diagram Explanation

The Class Diagram demonstrates the code structure. Since the API Developer has to develop the classes to assure encapsulation and relational integrity.

- **Classes (PostgreSQL):** The following classes (User, Job, Deliverable) are our relational database. I also have attracted Foreign Key constraints to ensure that a "Deliverable" cannot exist without a reference to a legitimate "Job."
- **Bridge Class (EscrowTransaction):** This class is the core of our hybrid system. This **is not**

a money keeping class. Nevertheless, it is storing the Transaction Hash and Status. The processes of this type are an attempt to poll the blockchain in order to ensure that our data collection locally is in sync with that of the blockchain.

- Validation Schemas (Pydantic): These classes are "stateless." These are only there to authenticate the structure of the information as it flows through the request of the user to our backend logic, so that our backend can be resistant to user error.

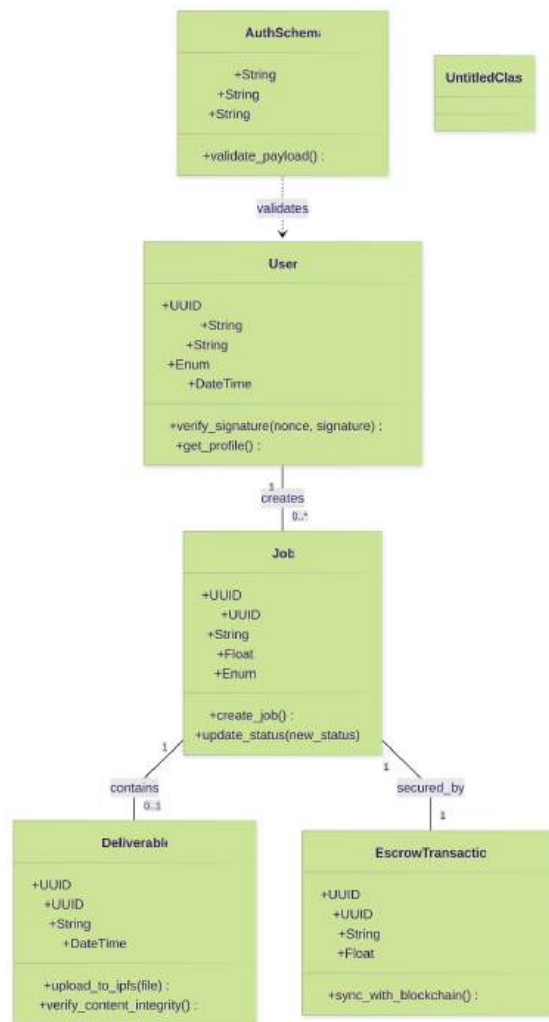


Fig: Class diagram for decentralized Freelance Protocol with Web3 Integration web3

4.7 Activity Diagram

This diagram illustrates the manner in which this system operates through the dynamic movement of data through logic gates in the system. The value of this number means that the backend is asynchronous.

- **Branching of Logic:** The diagram below illustrates the Pydantic Validation step. To ensure that dirty data is not written to the database, in the event that the data sent (e.g., wallet address, file buffer) are not as per the strict schema constraints, the activity is killed immediately with a 422 error to make sure that no dirty data is written in the database.
- **Parallel Processing:** Parallel processing is one of these aspects that are used here. The API will start working with the database using PostgreSQL during the hashing and pinning of the file to the Kubo Node. This ensures that we remain within the threshold of performance through providing high throughput despite high network congestions.
- **Final State:** This activity will not be finished until successful storage of Digital Wax Seal (CID) is completed and a confirmation token is issued to the frontend.

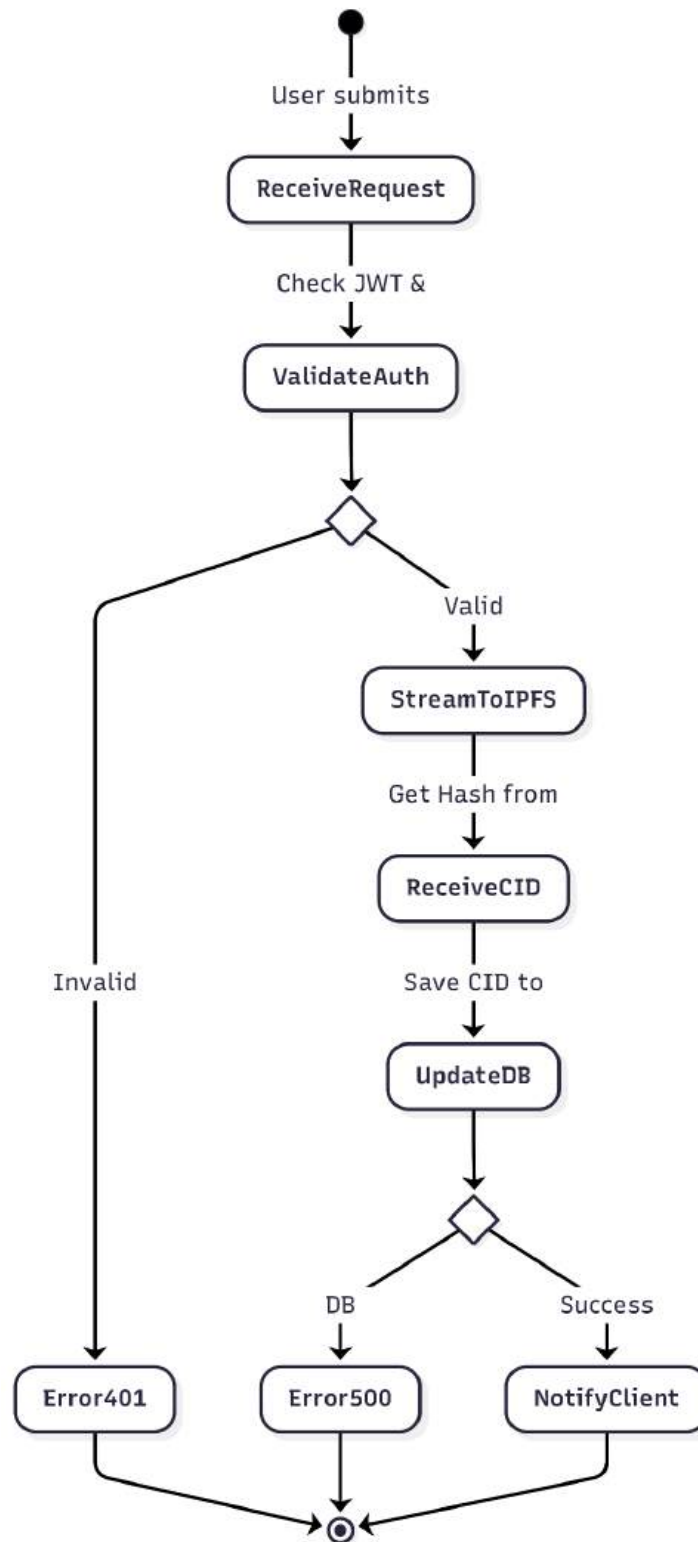


Fig: Activity diagram for decentralized Freelance Protocol with Web3

Integration web3

4.8 Sequence Diagram

Multiple sequence diagrams for the various functionalities of the created dApp(decentralised application) include:

4.8.1 Authentication & Registration Flow (The "Digital Wax Seal")

This flow substitutes the traditional centralized sessions with the Sovereign Identity verification.

- **Nonce Generation:** The user sends the FastAPI backend a request to generate a unique nonce that has time sensitivity so that it cannot be revisited.
- **Signature Recovery:** Signing this nonce can be done with the help of MetaMask and the web3.py library is used by the API to retrieve the public address of the signature.
- **Stateless Session:** When the address recovered is equal to the record in the database, the API sends a JWT (JSON Web Token) and the session can be secure, passwordless and with a latency of less than 1.5s.

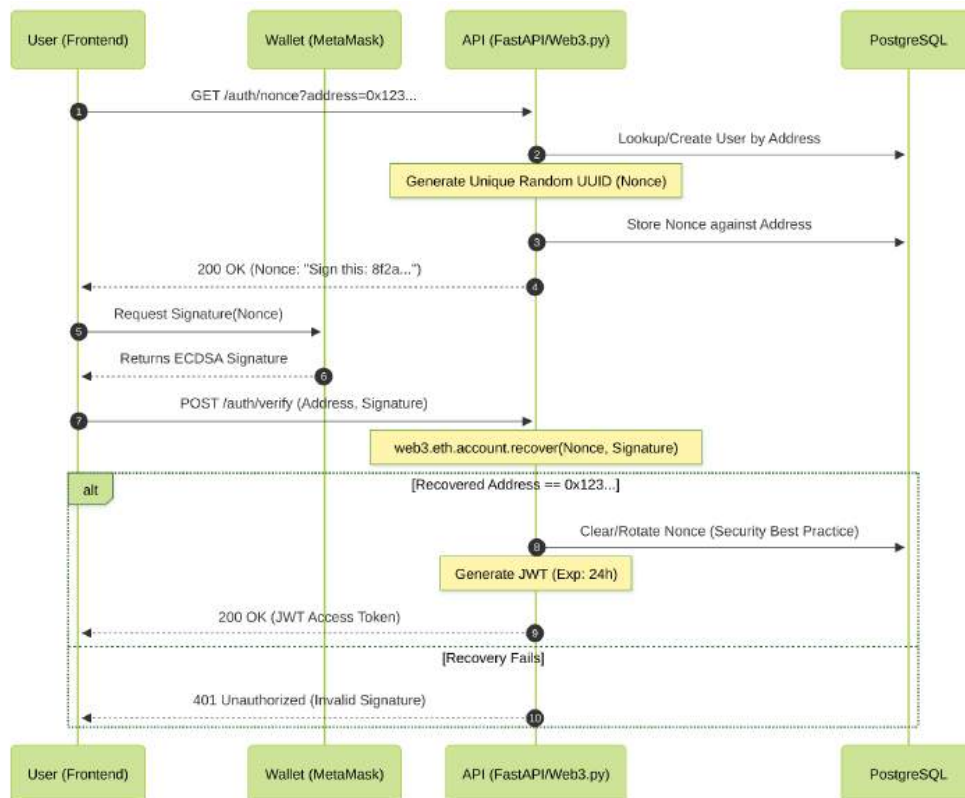


Fig:Authentication & Registration Flow (The "Digital Wax Seal")

4.8.2 Job Posting & Metadata Indexing Flow

This flow illustrates the use of the Hybrid Model in which the high-speed searchable data is processed using the API.

- Schema validation: The information on the job (title, budget, requirements) will be processed in real-time and subjected to type safety validation by Pydantic models before it is transferred to the persistence layer.
- Relational Indexing: A PostgreSQL transaction will be utilized to store the job metadata and a unique UUID will be used to trace the internal ID.
- Blockchain Reference: A confirmation will be returned to the frontend, and this is where the flow will be completed with on-chain escrow deposit being initiated with the distinct Job ID issued by the backend.

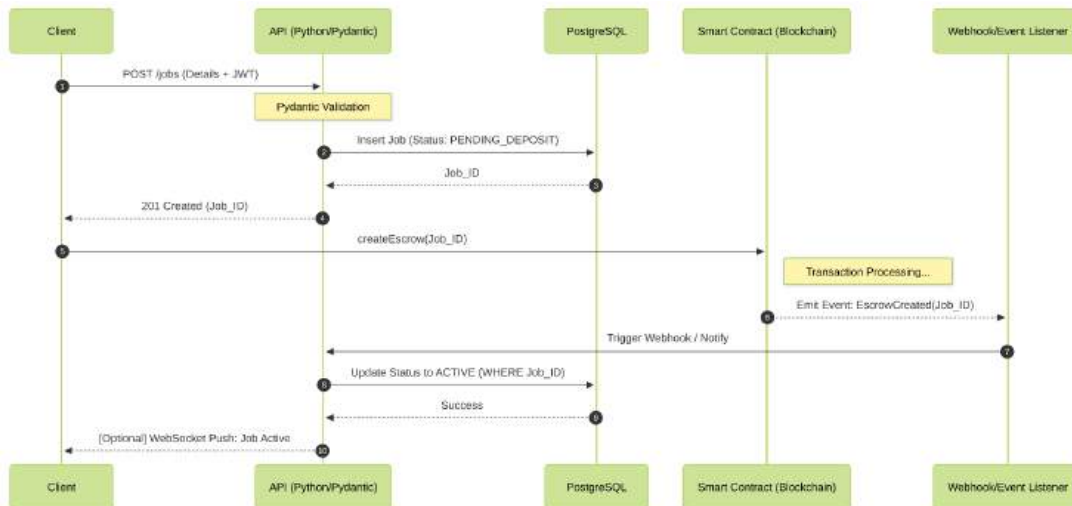


Fig: Job Posting & Metadata Indexing Flow

4.8.3 Deliverable Upload & IPFS

This is what the data engineering role entails, to work on the processing of raw files into Content-Addressable Storage.

- Stream Processing: The file buffer will be used to provide data to the API and forward it to the Kubo IPFS node instead of storing it in local disk in a bid to ensure that it works.
- CID Generation: As soon as the Kubo node is struck by a Content Identifier (CID), the API receives it as a permanent "Digital Receipt" of the work done.

- Atomic Commitment: PostgreSQL is further cross-linked to the deliverables table. This will make sure that the relational database will never refer to the distorted version of the file on the decentralized net.

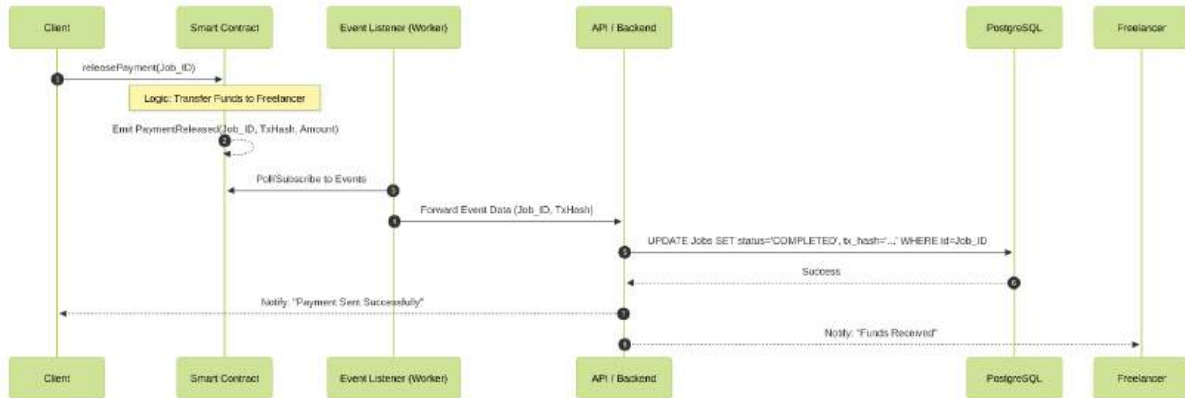


Fig: Deliverable Upload & IPFS

4.8.4 Payment Release & State Sync

This flow takes care of the State Synchronization between the Hardhat blockchain and the local metadata.

- Event Listening: The backend has a background worker that is asynchronous and listens to particular smart contract events, i.e. PaymentReleased.
- On-Chain Truth: When an event is observed on the Hardhat node, an API obtains the transaction hash (tx_hash) and the Job ID.
- Finality Sync: The API writes the job status to PostgreSQL to the state of PAID and records the tx_hash to audit. This makes sure that the user interface is showing the true financial status of the blockchain without having to refresh the interface manually.

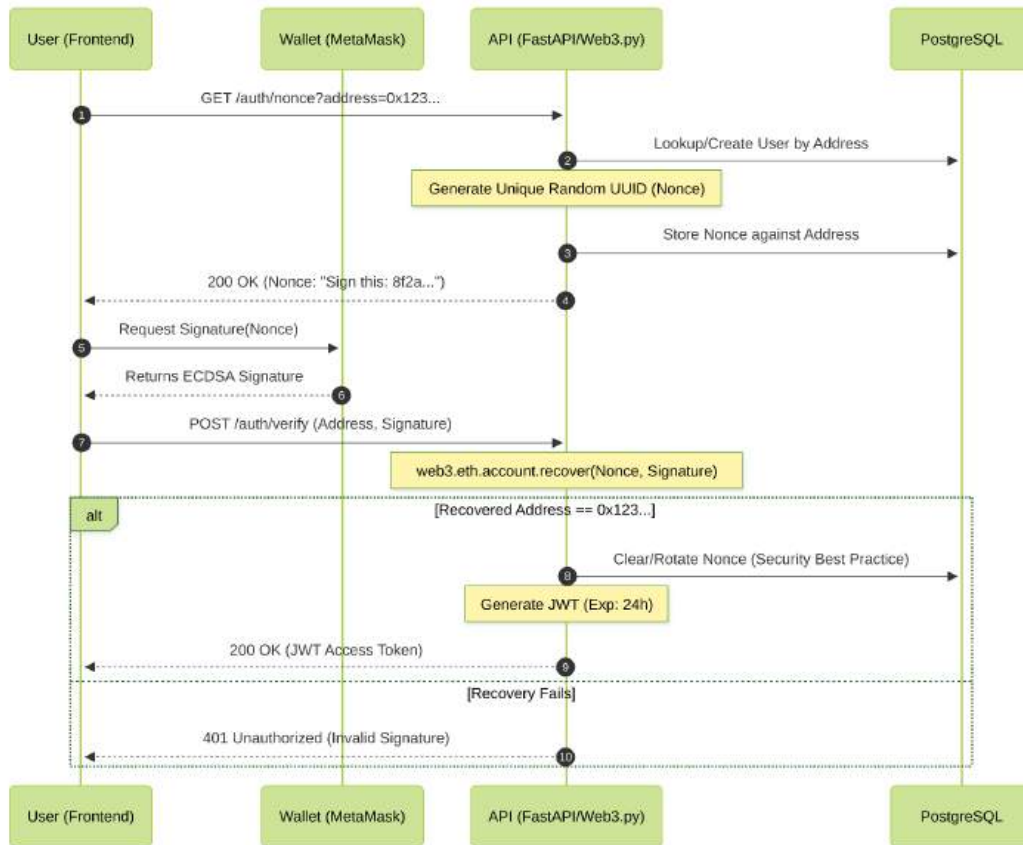


Fig: Payment Release & State Sync

4.8 Pseudocode and Algorithmic Structure

4.8.1 Core Functionality: ECDSA Authentication Logic

This algorithm replaces traditional password login. It uses cryptographic signature recovery to verify a user's "Sovereign Identity" at the API gateway.

```
# ALGORITHM: Verify Wallet Signature (Digital Wax Seal)
# INPUT: wallet_address, signature, nonce
# OUTPUT: JWT Access Token or Unauthorized Error

FUNCTION authenticate_user(wallet_address, signature, nonce):
  TRY:
    # 1. Use Web3 utility to recover the public address from the signature
    recovered_address = web3_recover_address(nonce, signature)

    # 2. Compare recovered address with the claimed wallet address
    IF recovered_address.toLowerCase() EQUALS wallet_address.toLowerCase():

      # 3. Check if user exists in PostgreSQL
      user_record = DB_QUERY("SELECT * FROM users WHERE wallet = ?",
wallet_address)

      IF user_record IS EMPTY:
        DB_EXECUTE("INSERT INTO users (wallet, created_at) VALUES (?, NOW())",
wallet_address)

      # 4. Generate a stateless session token
      token = generate_jwt({"sub": wallet_address, "exp": 3600})
      RETURN status_code(200), token

    ELSE:
      RETURN status_code(401), "Signature Verification Failed"

  EXCEPT Exception as e:
    RETURN status_code(400), e.message
```

4.8.2 Database Transactions: IPFS Metadata Mapping

This pseudocode outlines the atomic process of linking a physical file to a relational database record using its Content Identifier (CID).

```
# ALGORITHM: Store Deliverable with IPFS Handshake
# INPUT: file_data, job_id, user_token
# OUTPUT: CID and Success Message

FUNCTION process_project_submission(file_data, job_id, user_token):
    # 1. Verify Session through JWT
    user_wallet = decode_jwt(user_token)

    # 2. Check if user is the assigned freelancer for this Job_ID
    is_assigned = DB_QUERY("SELECT 1 FROM jobs WHERE id = ? AND freelancer = ?",
    job_id, user_wallet)

    IF NOT is_assigned:
        RETURN ERROR "Unauthorized access to this job"

    # 3. Stream file to Kubo (IPFS) Node
    # This creates the immutable 'Content-Address'
    ipfs_response = IPFS_NODE_ADD(file_data)
    generated_cid = ipfs_response.hash

    # 4. Atomic Transaction: Update PostgreSQL Metadata
    # We use a transaction to ensure data integrity
    BEGIN TRANSACTION:
        DB_EXECUTE("UPDATE jobs SET status = 'SUBMITTED' WHERE id = ?", job_id)
        DB_EXECUTE("INSERT INTO deliverables (job_id, cid, timestamp) VALUES (?, ?,
    NOW())", job_id, generated_cid)
    COMMIT

    RETURN SUCCESS, generated_cid

RETURN SUCCESS, generated_cid
```

4.8.3 Logic for State Synchronization (Blockchain Listener)

This logic ensures that your centralized PostgreSQL database stays in sync with the Hardhat blockchain events.

```
# ALGORITHM: Blockchain Event Synchronization
# INPUT: Blockchain Event Log
# OUTPUT: Updated Database State

FUNCTION sync_blockchain_event(event_payload):
  # 1. Extract data from the Smart Contract event
  job_id = event_payload.args.jobId
  tx_hash = event_payload.transactionHash
  event_type = event_payload.event # e.g., 'PaymentReleased'

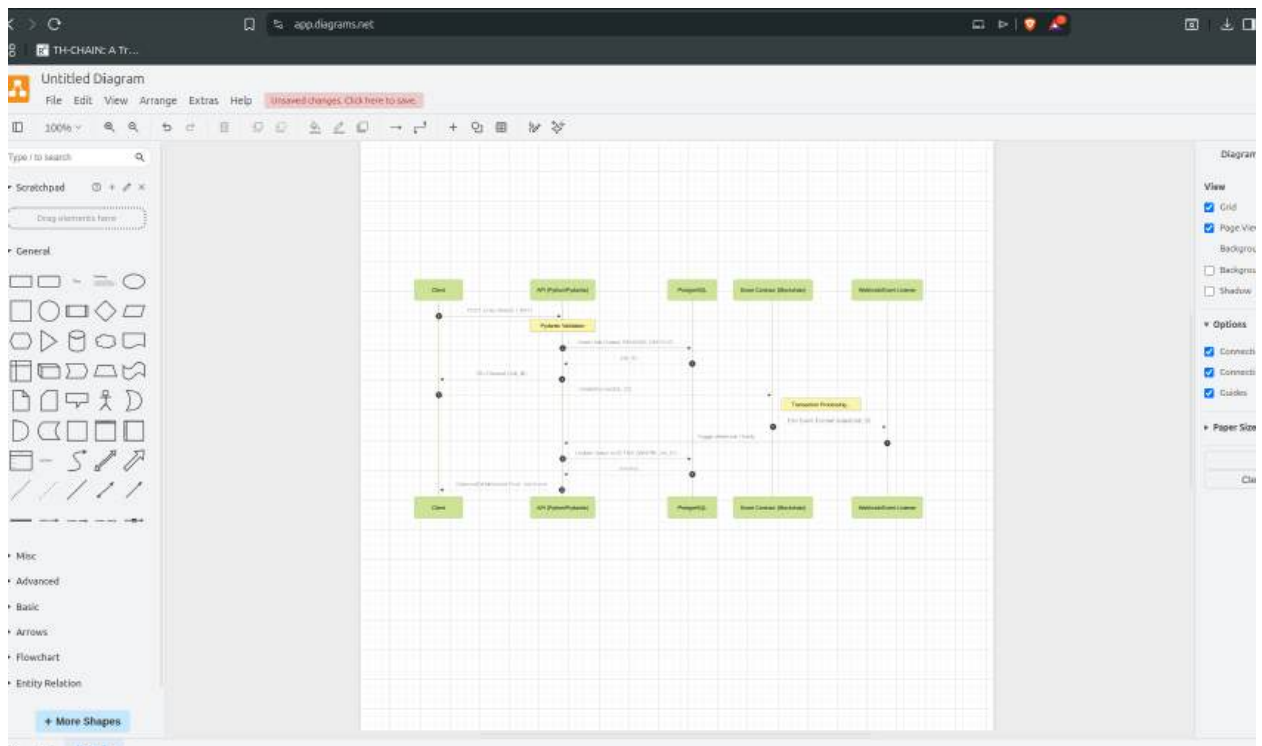
  # 2. Update PostgreSQL to reflect the 'On-Chain' truth
  IF event_type EQUALS "PaymentReleased":
    DB_EXECUTE("UPDATE jobs SET status = 'PAID', tx_hash = ? WHERE id = ?", tx_hash,
job_id)
    PRINT "Database synced: Job " + job_id + " is now PAID."

  ELSE IF event_type EQUALS "EscrowCreated":
    DB_EXECUTE("UPDATE jobs SET status = 'ACTIVE' WHERE id = ?", job_id)

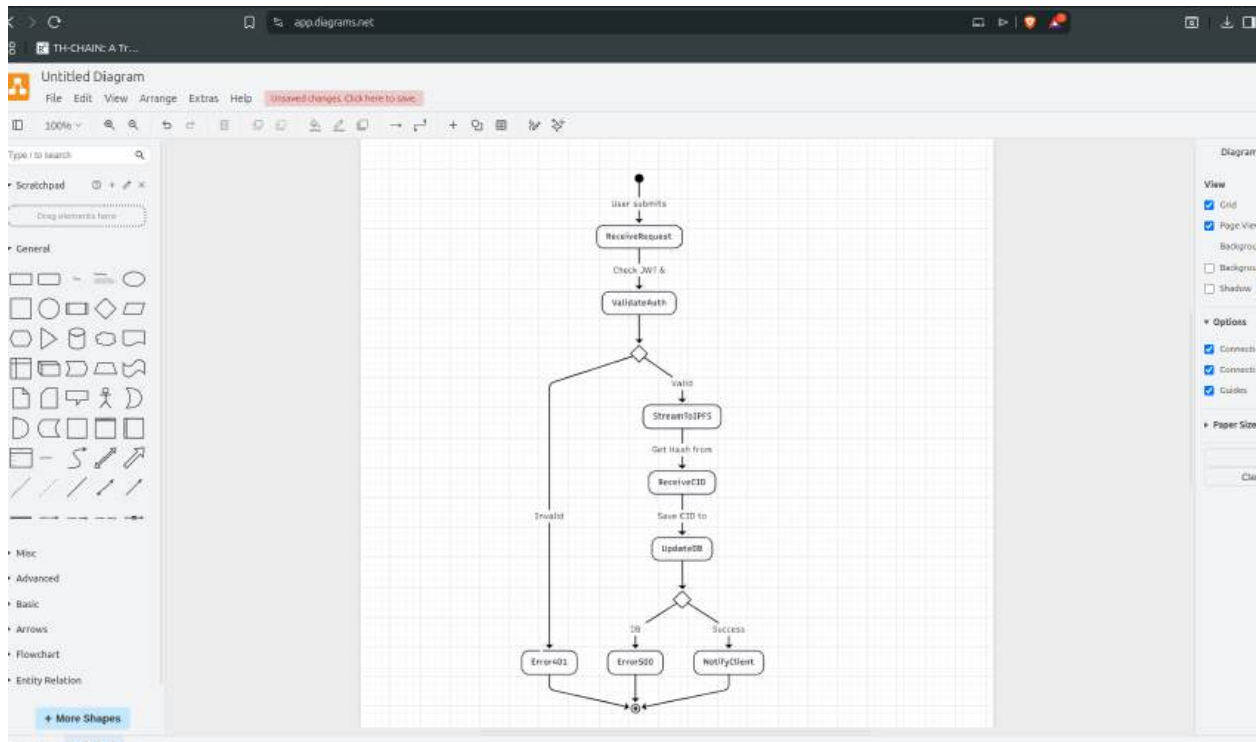
  # 3. Trigger Real-time Notification to Frontend
  WS_NOTIFY_USER(job_id, "Status Updated: " + event_type)
```

Appendices

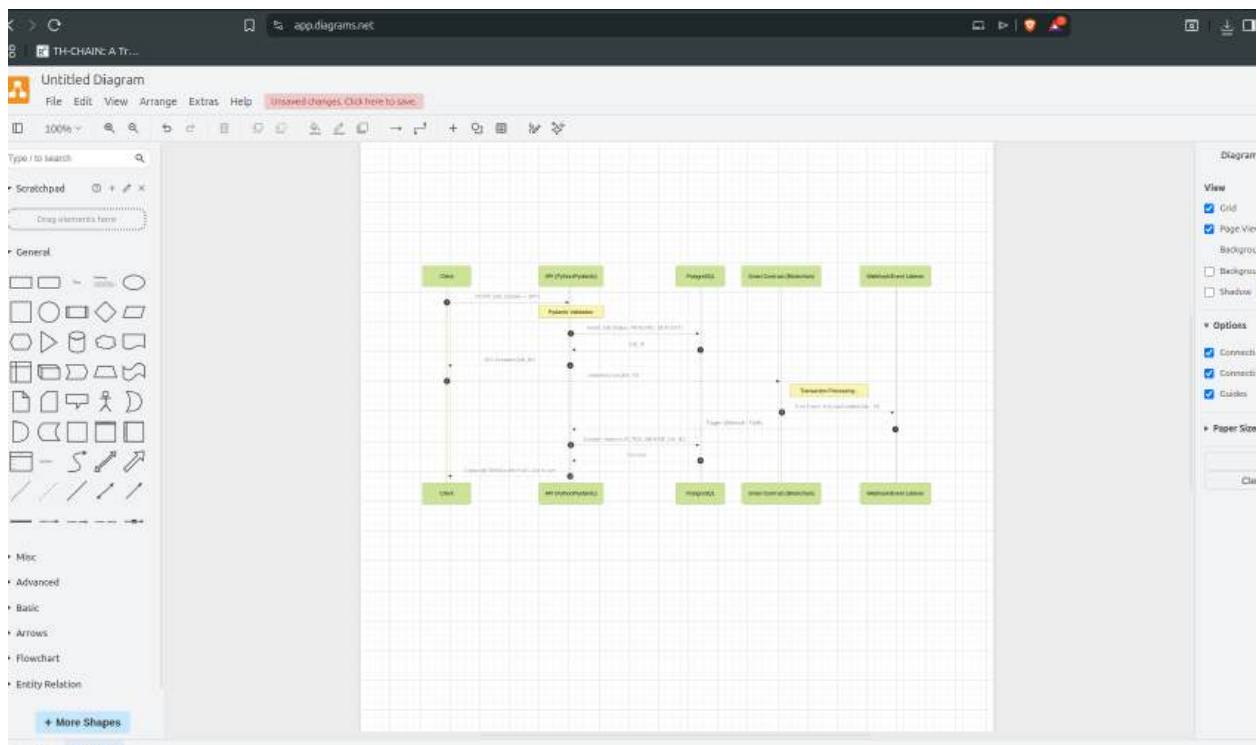
Screenshots



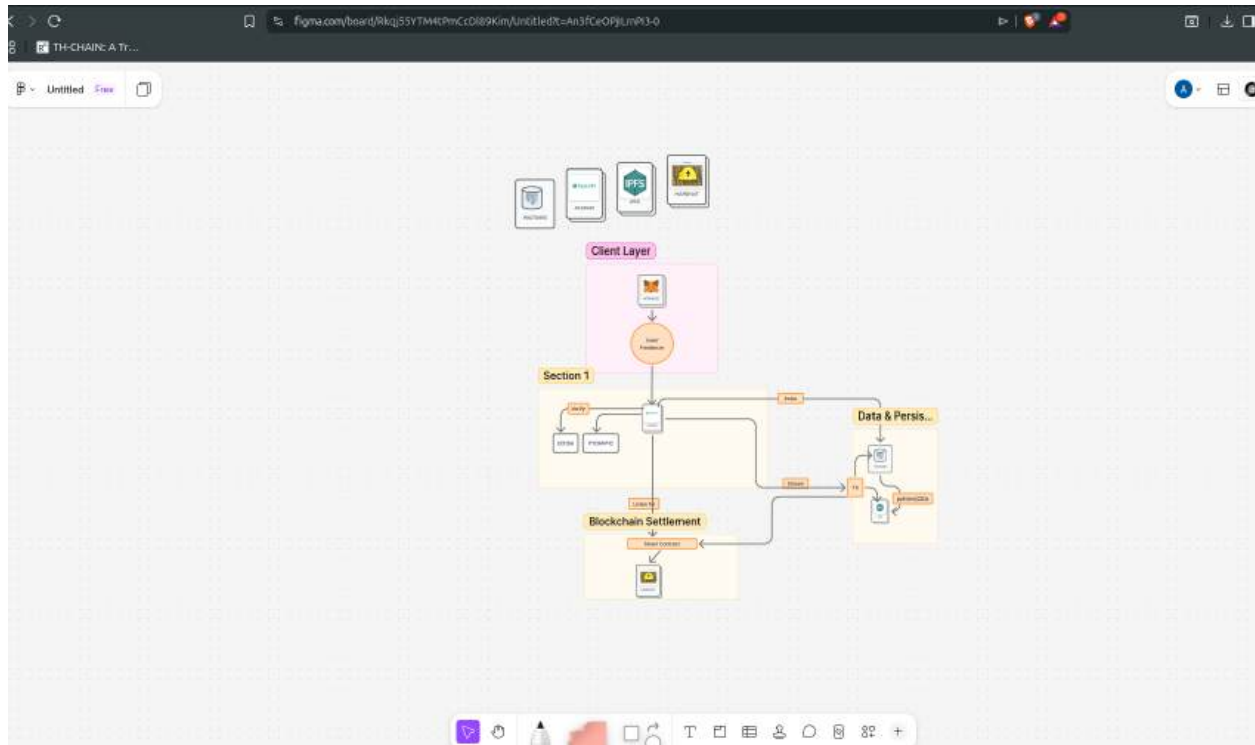
Sequence diagrams draw.io



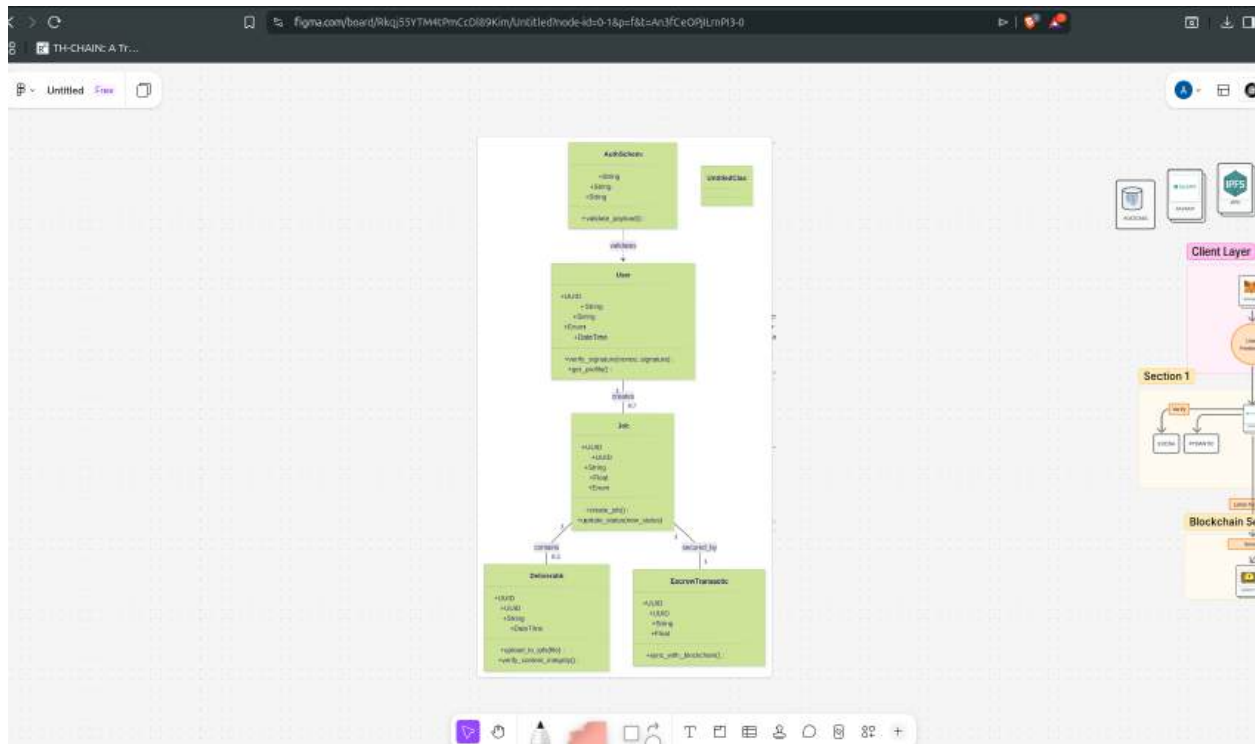
Activity diagram in draw.io



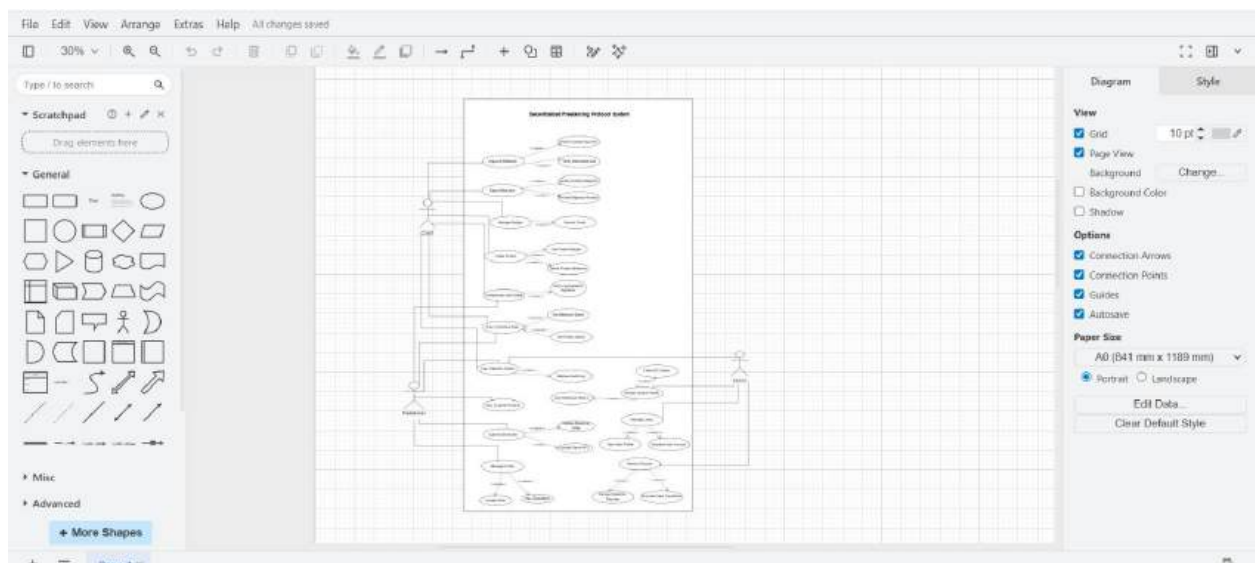
Sequence diagrams draw.io



System Architecture in figjam



Class diagram in figjam



Usecase Diagram in Draw.io

Meeting Minutes

Bachelor of Computer Science (Hons)

Capstone Project Log Sheet

Date: 30th March 2026

Group: 19

Meeting No: 7

Project Title: A Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Mr. Subit Timalsina

Supervisor's Signature: 

Item for Discussion: (Noted by Student Before Supervisory meeting)

- Refinement of Role-Specific UML Diagrams and Individual Report

Record of Discussion: (Noted by Student During Supervisory meeting)

- The supervisor requested that UML diagrams (Use Case, Activity, etc.) be refined according to specific roles within the team.
- Emphasized making the report "less AI" by adding deeper technical justifications for stack choices.

Action List (To be attempted or Completed by Student by the Next Supervisory meeting)

- Finalize the **Class Diagram** for the Backend and API infrastructure.

Bachelor of Computer Science (Hons)

Capstone Project Log Sheet

Date: 1st April 2026

Group: 19

Meeting No: 8

Project Title: A Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Mr. Subit Timalsina

Supervisor's Signature: 

Item for Discussion: (Noted by Student Before Supervisory meeting)

- Reviews for the everyone report.

Record of Discussion: (Noted by Student During Supervisory meeting)

- Report formatting

Action List (To be attempted or Completed by Student by the Next Supervisory meeting)

Completion of the report for AI check.

Bachelor of Computer Science (Hons)

Capstone Project Log Sheet

Date: 31st March 2026

Group: 19

Meeting No: 7

Project Title: A Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Mr. Subit Timalsina

Supervisor's Signature: 

Item for Discussion: (Noted by Student Before Supervisory meeting)

- What to do in the pseudocode.
- Reviews for everyone's report.

Record of Discussion: (Noted by Student During Supervisory meeting)

- Since the use-case and UML diagram is already done, now you need to make similar other diagrams according to your roles.
- Refine the report more and make less AI.

Action List (To be attempted or Completed by Student by the Next Supervisory meeting)

- Progress on diagrams and pseudocodes of everyone.
- Completion of the report for AI check.

Bachelor of Computer Science (Hons)

Capstone Project Log Sheet

Date: 10th March 2026

Group: 19

Meeting No: 1

Project Title: A Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Mr. Subit Timalsina

Supervisor's Signature: 

Item for Discussion: (Noted by Student Before Supervisory meeting)

- How to start and brainstorm.

Record of Discussion: (Noted by Student During Supervisory meeting)

- Find a relevant paper and start literature review

Action List (To be attempted or Completed by Student by the Next Supervisory meeting)

- Research existing freelance platforms (Fiverr/Upwork) and identify technical gaps.

Bachelor of Computer Science (Hons)

Capstone Project Log Sheet

Date: 29th March 2026

Group: 19

Meeting No: 5

Project Title: A Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Mr. Subit Timahina

Supervisor's Signature:

Item for Discussion: (Noted by Student Before Supervisory meeting)

- Review of literature review
- Start doing UML diagrams

Record of Discussion: (Noted by Student During Supervisory meeting)

- Literature is fine.
- Multiple Sequence diagram for multiple flows

Action List (To be attempted or Completed by Student by the Next Supervisory meeting)

- Progress on diagrams .

Bachelor of Computer Science (Hons)

Capstone Project Log Sheet

Date: 27th March 2026

Group: 19

Meeting No: 4

Project Title: A Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Mr. Subit Timahina

Supervisor's Signature:

Item for Discussion: (Noted by Student Before Supervisory meeting)

- Literature review's review

Record of Discussion: (Noted by Student During Supervisory meeting)

- Current one is fine, find more papers

Action List (To be attempted or Completed by Student by the Next Supervisory meeting)

- Adding more papers and justifications in the literature review

Bachelor of Computer Science (Hons)

Capstone Project Log Sheet

Date: March 18, 2026

Group: 19

Meeting No: 2

Project Title: A Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Mr. Subit Timahina

Supervisor's Signature:

Item for Discussion: (Noted by Student Before Supervisory meeting)

- Literature Review Findings.

Record of Discussion: (Noted by Student During Supervisory meeting)

- Formally assigned my role as **Backend & API Developer**, focusing on building the high-speed bridge between users and the ledger.
- Discussed the "Hybrid Model" (PostgreSQL + Blockchain) as a potential solution found in recent literature.

Action List (To be attempted or Completed by Student by the Next Supervisory meeting)

- Summarize key findings from Tamsawat et al. and Singh et al. for the proposal.

Bachelor of Computer Science (Hons)

Capstone Project Log Sheet

Date: 24th March 2026

Group: 19

Meeting No: 3

Project Title: A Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Mr. Subit Timahina

Supervisor's Signature:

Item for Discussion: (Noted by Student Before Supervisory meeting)

- Literature Review

Record of Discussion: (Noted by Student During Supervisory meeting)

- Since the literature review lacks some of the key elements that is related to backend API and centralised Database , rules.

Action List (To be attempted or Completed by Student by the Next Supervisory meeting)

- Redo and find more papers

References

Boonchieng, W., et al. (2025). Super APIs for blockchain apps: High-performance financial transaction design in DApps. *Journal of Cloud Computing*,

<https://jit.ndhu.edu.tw/article/view/3235>

Careja, V., et al. (2026). Digital identity using blockchain: A hierarchical tokenized ecosystem for government-certified identity. *Applied Sciences*

https://www.erpublications.com/uploaded_files/download/v-kadam-v-kabade-t-dalavi-a-pisule-d-saisudha_mzyFX.pdf

Gupta, A., et al. (2024). Enhancing blockchain scalability with snake optimization algorithm: A novel approach. *Frontiers in Blockchain*

<https://www.frontiersin.org/journals/blockchain/articles/10.3389/fbloc.2024.1361659/full>

Hwang, J., et al. (2026). AI disclosure in freelance work: Implications for frontline service employees and future research. *Journal of Service Management*

<https://arxiv.org/pdf/2603.07459>

Kadam, S., et al. (2024). Analysis of decentralized freelancing: Two-way confirmation logic for task completion. *International Journal of Decentralized Systems*

https://www.erpublications.com/uploaded_files/download/v-kadam-v-kabade-t-dalavi-a-pisule-d-saisudha_mzyFX.pdf

Pippal, S. K., & Sharma, A. (2025). BBAS: A blockchain-based authentication system for e-health with multi-factor authentication and IPFS-based infrastructure. *Journal of Medical Systems*

Ren, X., et al. (2025). SensWorkflow: A high-performance framework for remote sensing big data processing on heterogeneous clusters. *Big Earth Data*

<https://scispace.com/pdf/indexfs-scaling-file-system-metadata-performance-with-2hspv1ainw.pdf>

Singh, R., et al. (2025). AI-driven blockchain lending for sustainable development: A machine learning framework for loan risk and eligibility classification. *PeerJ Computer Science*,

<https://www.frontiersin.org/journals/blockchain/articles/10.3389/fbloc.2025.1495984/full>

Singh, V., et al. (2025). A hybrid blockchain and smart contract framework for resilient IoT security in smart homes. *Frontiers in Blockchain*

https://www.researchgate.net/publication/396888772_Trustless_Marketplaces_Powered_by_AI_and_Blockchain_Integration

Singh, P., et al. (2025). Securing e-governance: A blockchain-based framework for tamper-proof document exchange. *Frontiers in Blockchain*

<https://sarcouncil.com/download-article/SJECS-486-2025-418-424.pdf>

Sharma, A., & Gupta, R. (2025). Evolution and future trends in web development: A comprehensive review of non-blocking I/O in decentralized architectures. *Journal of Modern Software Engineering*

https://www.researchgate.net/publication/389902743_Evolution_and_Future_Trends_in_Web_Development_A_Comprehensive_Review